

torch.fft.ihfftn#

<https://pytorch.org/docs/stable/generated/torch.fft.ihfftn.html>

Description:

Computes the N-dimensional inverse discrete Fourier transform of real input. input must be a real-valued signal, interpreted in the Fourier domain. The n-dimensional IFFT of a real signal is Hermitian-symmetric, $X[i, j, \dots] = \text{conj}(X[-i, -j, \dots])$. `ihfftn()` represents this in the one-sided form where only the positive frequencies below the Nyquist frequency are included in the last signal dimension. To compute the full output, use `ifftn()`. Supports `torch.half` on CUDA with GPU Architecture SM53 or greater. However it only supports powers of 2 signal length in every transformed dimensions. input (Tensor) – the input tensor

Function Signature

```
torch.fft.ihfftn(input, s=None, dim=None, norm=None, *,  
out=None) → Tensor#
```

Parameters

Keyword Arguments

Parameters

Keyword

out (Tensor, optional) – the output tensor.

Code Examples

```
>>> T = torch.rand(10, 10)
>>> ihfftn = torch.fft.ihfftn(T)
>>> ihfftn.size()
torch.Size([10, 6])
```

```
>>> ifftn = torch.fft.ifftn(t)
>>> torch.allclose(ifftn[..., :6], ihfftn)
True
```

```
>>> two_iffts = torch.fft.ifft(torch.fft.ihfft(t, dim=1), dim=0)
>>> torch.allclose(ihfftn, two_iffts)
True
```

Notes & Warnings

NOTE

Note Supports torch.half on CUDA with GPU Architecture SM53 or greater. However it only supports powers of 2 signal length in every transformed dimensions.

Detailed Documentation

Documentation

Computes the N-dimensional inverse discrete Fourier transform of real input.

input must be a real-valued signal, interpreted in the Fourier domain. The n-dimensional IFFT of a real signal is Hermitian-symmetric, $X[i, j, \dots] = \text{conj}(X[-i, -j, \dots])$. `ihfftn()` represents this in the one-sided form where only the positive frequencies below the Nyquist frequency are included in the last signal dimension. To compute the full output, use `ifftn()`.

Supports torch.half on CUDA with GPU Architecture SM53 or greater. However it only supports powers of 2 signal length in every transformed dimensions.

input (Tensor) – the input tensor

s (Tuple[int], optional) – Signal size in the transformed dimensions. If given, each dimension `dim[i]` will either be zero-padded or trimmed to the length `s[i]` before computing the Hermitian IFFT. If a length -1 is specified, no padding is done in that dimension. Default: `s = [input.size(d) for d in dim]`

`dim (Tuple[int], optional)` – Dimensions to be transformed. Default: all dimensions, or the last `len(s)` dimensions if `s` is given.

`norm (str, optional)` –

Normalization mode. For the backward transform (`ihfft()`), these correspond to:

"forward" - no normalization

"backward" - normalize by $1/n$

"ortho" - normalize by $1/\sqrt{n}$ (making the Hermitian IFFT orthonormal)

Where $n = \text{prod}(s)$ is the logical IFFT size. Calling the forward transform (`hfft()`) with the same normalization mode will apply an overall normalization of $1/n$ between the two transforms. This is required to make `ihfft()` the exact inverse.

Default is "backward" (normalize by $1/n$).

`out (Tensor, optional)` – the output tensor.

Compared against the full output from `ifft()`, we have all elements up to the Nyquist frequency.

The discrete Fourier transform is separable, so `ihfft()` here is equivalent to a combination of `ihfft()` and `ifft()`: